

Problem

Offline goal-conditioned RL must reach **arbitrary goals** from a fixed dataset under **sparse binary rewards**. Most methods lean on a learned **value function**, but here it is **fragile**: out-of-distribution actions corrupt the estimates, and these errors compound over time until the policy diverges.

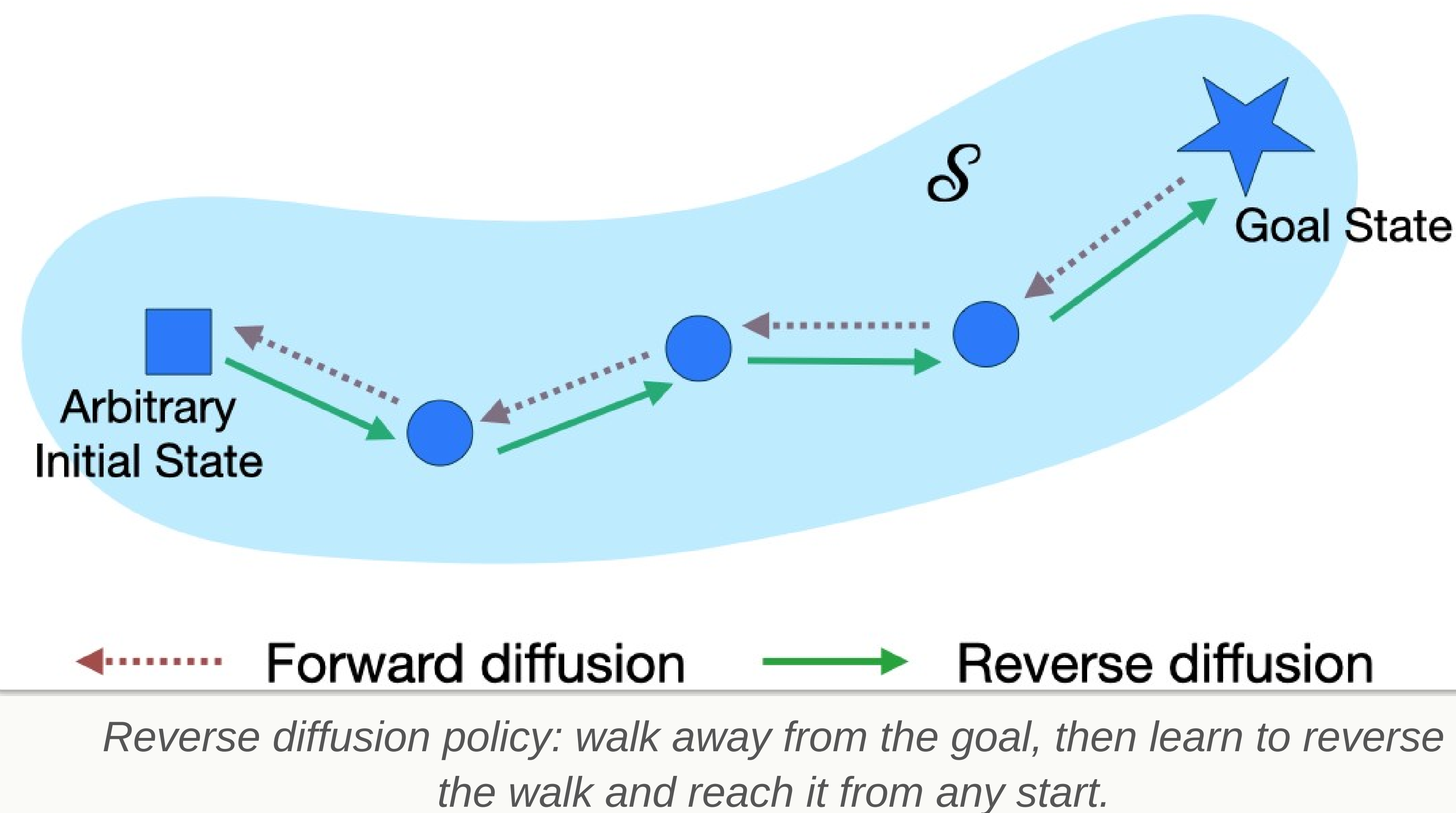
No reliable value function $\Rightarrow$ offline goal-reaching stays unstable.

Motivation

- Diffusion models learn a target distribution **without any value function**, by reversing a process that gradually turns clean data into pure noise.
- Cast goal states as that data distribution: build trajectories that **walk away** from goals, then learn to **reverse** them, directly analogous to learning the score.

Reversing a goal-departing walk sidesteps value estimation entirely.

Goal-conditioned Reinforcement Learning

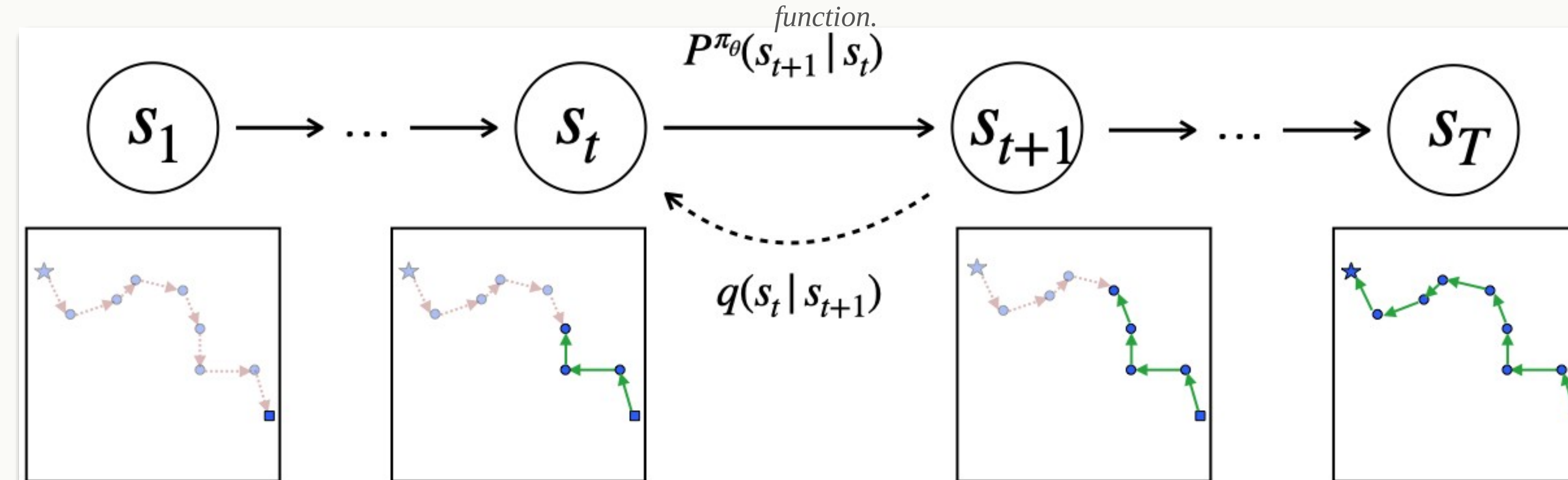


Method

- Read an offline trajectory **backwards** from its goal; a forward diffusion transform pushes states progressively **away** from the goal, like adding noise.
- Train a goal-conditioned policy to **reverse** the drift. At inference it needs only a **single denoising step** per environment step.
- Three forward processes**: a fixed rule (**Merlin**), a learned parametric model (**Merlin-P**), and **Merlin-NP**, which stitches nearest neighbors in a learned latent space.

$$\theta^* = \arg \max_{\theta} \mathbb{E}_{g, (s,a) \sim \mathcal{D}(g)} [\log \pi_{\theta}(a | s, g)]$$

Goal-conditioned behavior cloning maximizes a lower bound on the goal log-likelihood, with no value function.



Forward (red) pushes states away from the goal (star); the learned reverse policy (green) walks them back, shown on 2D navigation.

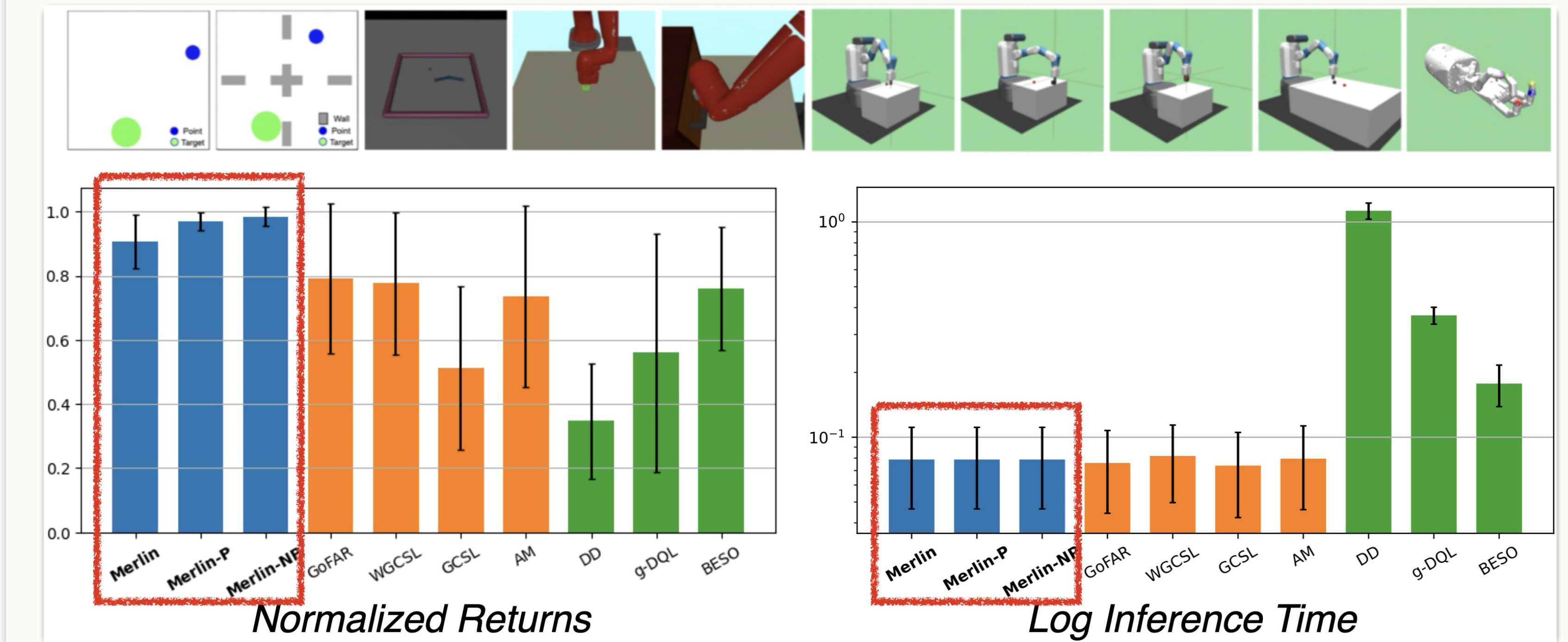
Dataset / Benchmark

- Benchmark of Yang et al. (2021): **10 control tasks**, sparse binary reward, trajectories capped at length **T = 50**, all averaged over 10 seeds.
- Two regimes per task, **expert** and **random**, in both **state** and **pixel** observation spaces.

PointReach PointRooms Reacher SawyerReach SawyerDoor
 FetchReach FetchPush FetchPick FetchSlide HandReach

Key Results

Merlin-NP: best overall rank, ~10× faster.



Returns (left) and log inference time (right): Merlin variants (blue) beat baselines while running ~10× faster.

Headline Numbers

~10×

Faster end-to-end training & inference than other diffusion-based reinforcement learning methods

Just one denoising step per environment step, versus many for prior diffusion policies

1.7 avg rank, state (of 10)

1.25 avg rank, pixel (of 10)

10 tasks · 10 seeds

Takeaway

Reframing goal-conditioned RL as **reverse diffusion** turns goal-reaching into simple, **value-free behavior cloning** that matches or beats state of the art, far more efficiently.

SO WHAT

Diffusion **in the state space**, with one denoising step per environment step, makes diffusion for RL both simple and scalable.